

Lab 3: Architectural Pattern Selection — Justification

Problem Statement #43 — Feature Flag & Dynamic Config Manager

Architecture Selection: *"We chose Microservices Architecture for the Feature Flag & Dynamic Config Manager System."*

Reason 1 — Independent Scaling for Latency-Critical Evaluation

NFR-001 requires the flag evaluation API to respond in under 15 ms, even at peak load. A microservices style lets the Flag Evaluation Service be scaled horizontally and independently of lower-traffic services such as Audit & Logging or the Admin Dashboard, so compute is spent where the latency budget is tightest instead of over-provisioning the whole system.

Reason 2 — Independent Deployment & Fault Isolation

Release Managers frequently push rollout-percentage and environment-override changes through the Config Management Service. Microservices allow this service to be redeployed on its own, without restarting the Flag Evaluation Service or dropping the persistent connections that the Real-Time Sync Service holds with client SDKs. Likewise, if the Audit & Logging Service fails, it does not bring down live flag evaluation for connected clients.

Security Advantage

Authentication and role-based access control (Software Engineer vs. Release Manager permissions) are centralized in a dedicated Authentication & Access Control Service issuing short-lived tokens. Every other service validates a token through this single required interface rather than handling credentials itself, so a compromise of the Flag Evaluation Service or Real-Time Sync Service cannot expose the credential store, and permission changes only need to be enforced in one place.

Performance Benefit

Decoupling flag evaluation from configuration storage lets the Flag Evaluation Service keep an in-memory cache of the latest config snapshot (kept current by the Real-Time Sync Service) instead of querying a database on every request. Removing that synchronous database call from the read path, combined with the ability to add replicas independently, is what allows the service to consistently meet the sub-15 ms evaluation target under peak load.